TikuKits ML: Linear Regression for Embedded Systems

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 14, 2026

Outline

1. Motivation
2. Architecture
3. Linear Regression Theory
4. Data Structure
5. Return Codes
6. API Reference
7. Configuration
8. Implementation Details
9. Examples
10. Test Suite
11. File Organization
12. Summary

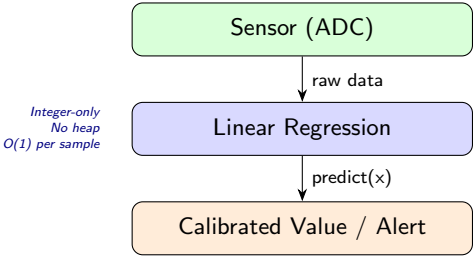
Why ML on Ultra-Low-Power MCUs?

Embedded Use Cases

- **Sensor calibration** — map raw ADC to physical units
- **Trend detection** — detect drift in readings over time
- **Prediction** — estimate battery charge from voltage
- **Anomaly flagging** — R^2 drop signals sensor failure

Constraints

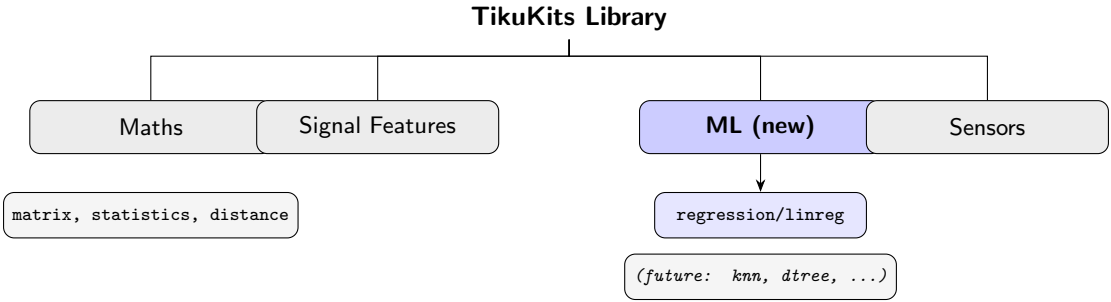
- No FPU (16-bit MSP430)
- No heap allocation
- Limited RAM (2–8 KB)
- Every µJ counts



Design Goal

Bring linear regression to devices with **no floating point**, using only static memory and integer arithmetic.

ML Kit in the TikuKits Ecosystem



File	Role
`ml/tiku_kits_ml.h`	Common return codes for all ML sub-modules
`ml/regression/tiku_kits_ml_linreg.h`	Linear regression API
`ml/regression/tiku_kits_ml_linreg.c`	Implementation

Ordinary Least Squares (OLS)

Model: $y = \beta_1 x + \beta_0$

Given n data points (x_i, y_i), the OLS solution minimises $\sum (y_i - \hat{y}_i)^2$:

$$\beta_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - \left(\sum x_i\right)^2}$$

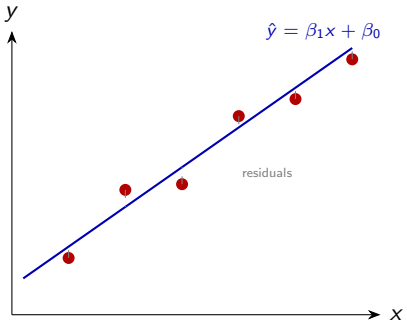
$$\beta_0 = \frac{\sum y_i - \beta_1 \sum x_i}{n}$$

Prediction:

$$\hat{y} = \beta_1 x + \beta_0$$

Goodness of fit:

$$R^2 = \frac{\left(nS_{xy} - S_x S_y\right)^2}{\left(nS_{xx} - S_x^2\right)\left(nS_{yy} - S_y^2\right)}$$



Key Insight

OLS only needs **five running sums**:
S_x, S_y, S_{xx}, S_{xy}, S_{yy}.
No sample buffer required — O(1) memory.

Integer-Only Implementation Strategy

Problem

Slope and intercept are typically **fractional**. MSP430 has no FPU — division truncates to zero.

$\beta_1 = 1.5 \rightarrow$ `int` gives 1 (33% error).

Solution: Fixed-Point Q-Format

Scale by 2^{shift} before dividing:

$$\text{slope_q} = \frac{\text{num} \ll \text{shift}}{\text{den}}$$

With `shift=8`, resolution $= \frac{1}{256} \approx 0.004$.

To recover real value: slope $= \frac{\text{slope_q}}{256}$.

Overflow Prevention

- All accumulators use `int64_t`
- Products $x \cdot y$ computed as `(int64_t)x * (int64_t)y`
- Safe for `int32_t` inputs: $2^{31} \times 2^{31} = 2^{62} < 2^{63}$

Prediction Without Scaling

`predict()` returns values in the **original integer domain** — no Q-format decoding needed:

$$\hat{y} = \frac{\text{num} \cdot (n \cdot x - S_x) + S_y \cdot \text{den}}{n \cdot \text{den}}$$

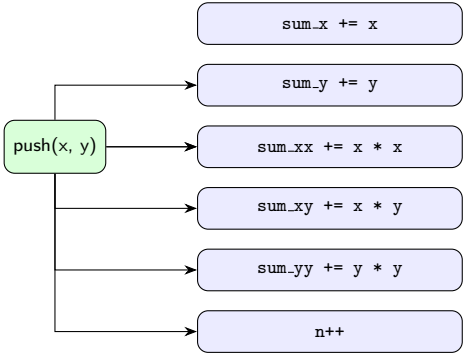
The tiku_kits_ml_linreg Structure

ml/regression/tiku_kits_ml_linreg.h

```
struct tiku_kits_ml_linreg {
    int64_t sum_x;    /* Sum of x        */
    int64_t sum_y;    /* Sum of y        */
    int64_t sum_xx;   /* Sum of x*x      */
    int64_t sum_xy;   /* Sum of x*y      */
    int64_t sum_yy;   /* Sum of y*y      */
    uint16_t n;       /* Data points     */
    uint8_t shift;    /* Q-format bits   */
};
```

Memory footprint:
$5 \times 8 + 2 + 1 = 43$ bytes per model.
All statically allocated — no heap.

Streaming accumulator pattern:



Each `push()` is **O(1)** — five multiply-add operations.

ML Kit Return Codes

ml/tiku_kits_ml.h

```
#define TIKU_KITS_ML_OK            0
#define TIKU_KITS_ML_ERR_NULL     (-1)
#define TIKU_KITS_ML_ERR_SIZE     (-2)
#define TIKU_KITS_ML_ERR_PARAM    (-3)
#define TIKU_KITS_ML_ERR_SINGULAR (-4)
#define TIKU_KITS_ML_ERR_OVERFLOW (-5)
```

Defined in the **kit-level common header** — shared by all ML sub-modules (linear regression, and future modules like kNN, decision tree, etc.).

Code	Meaning
OK	Operation succeeded
ERR_NULL	NULL pointer passed
ERR_SIZE	Fewer than 2 data points
ERR_PARAM	Invalid parameter (e.g. shift ¿ 30)
ERR_SINGULAR	All x values identical (zero variance)
ERR_OVERFLOW	Arithmetic overflow

Complete API

Function	Description
`tiku_kits_ml_linreg_init(lr, shift)`	Initialize model with Q-format precision (shift = 0..30)
`tiku_kits_ml_linreg_reset(lr)`	Clear all data points, preserve shift
`tiku_kits_ml_linreg_push(lr, x, y)`	Stream a data point — O(1), 5 multiply-adds
`tiku_kits_ml_linreg_slope(lr, &result)`	Fitted slope $\times 2^{\text{shift}}$ (Q-format)
`tiku_kits_ml_linreg_intercept(lr, &result)`	Fitted intercept $\times 2^{\text{shift}}$ (Q-format)
`tiku_kits_ml_linreg_predict(lr, x, &result)`	Predicted $\hat{y}$ in original integer scale
`tiku_kits_ml_linreg_r2(lr, &result)`	$R^2 \times 2^{\text{shift}}$; range $[0, 2^{\text{shift}}]$
`tiku_kits_ml_linreg_count(lr)`	Number of data points pushed

Minimum Data Requirement

`slope()`, `intercept()`, `predict()`, and `r2()` require ≥ 2 data points with distinct x values. Returns `ERR_SIZE` or `ERR_SINGULAR` otherwise.

Compile-Time Configuration

Configurable element type

```c
/* Default: int32_t (no FPU needed) */
#ifndef TIKU_KITS_ML_ELEM_TYPE
#define TIKU_KITS_ML_ELEM_TYPE int32_t
#endif

typedef TIKU_KITS_ML_ELEM_TYPE
    tiku_kits_ml_elem_t;

/* Override for tight memory: */
#define TIKU_KITS_ML_ELEM_TYPE int16_t
#include "tiku_kits_ml_linreg.h"
```

Configurable Q-format precision

```c
/* Default: 8 fractional bits */
#ifndef TIKU_KITS_ML_LINREG_SHIFT
#define TIKU_KITS_ML_LINREG_SHIFT 8
#endif

/* Or set at runtime: */
tiku_kits_ml_linreg_init(&lr, 12);
/* 12-bit: resolution ~0.00024 */
```

Shift	Scale	Resolution
4	16	0.0625
8	256	0.0039
12	4096	0.00024
16	65536	0.000015

Core: Push & Slope

tiku_kits_ml_linreg.c — push

```c
int tiku_kits_ml_linreg_push(
    struct tiku_kits_ml_linreg *lr,
    tiku_kits_ml_elem_t x,
    tiku_kits_ml_elem_t y)
{
    if (lr == NULL)
        return TIKU_KITS_ML_ERR_NULL;

    lr->sum_x  += (int64_t)x;
    lr->sum_y  += (int64_t)y;
    lr->sum_xx += (int64_t)x * (int64_t)x;
    lr->sum_xy += (int64_t)x * (int64_t)y;
    lr->sum_yy += (int64_t)y * (int64_t)y;
    lr->n++;

    return TIKU_KITS_ML_OK;
}
```

tiku_kits_ml_linreg.c — slope

```c
int tiku_kits_ml_linreg_slope(
    const struct tiku_kits_ml_linreg *lr,
    int32_t *result)
{
    int64_t num, den;
    int rc;

    if (lr == NULL || result == NULL)
        return TIKU_KITS_ML_ERR_NULL;

    rc = slope_parts(lr, &num, &den);
    if (rc != TIKU_KITS_ML_OK)
        return rc;

    /* Fixed-point scaling */
    *result = (int32_t)(
        (num << lr->shift) / den);
    return TIKU_KITS_ML_OK;
}
```

Core: Predict & R-Squared

predict — returns original scale

```c
int tiku_kits_ml_linreg_predict(
    const struct tiku_kits_ml_linreg *lr,
    tiku_kits_ml_elem_t x,
    tiku_kits_ml_elem_t *result)
{
    int64_t num, den, y_num;
    /* ... NULL/size checks ... */

    /*
     * y = (num*(n*x - Sx) + Sy*den)
     *     / (n * den)
     *
     * No fixed-point decoding needed.
     */
    y_num = num * ((int64_t)lr->n
                 * (int64_t)x - lr->sum_x)
            + lr->sum_y * den;

    *result = (tiku_kits_ml_elem_t)(
        y_num / ((int64_t)lr->n * den));
    return TIKU_KITS_ML_OK;
}
```

R-squared — goodness of fit

```c
int tiku_kits_ml_linreg_r2(
    const struct tiku_kits_ml_linreg *lr,
    int32_t *result)
{
    int64_t num, den, ss_yy;
    /* ... NULL/size checks ... */

    /* SS_yy = n*Syy - Sy^2 */
    ss_yy = (int64_t)lr->n * lr->sum_yy
            - lr->sum_y * lr->sum_y;

    if (ss_yy == 0)
        return TIKU_KITS_ML_ERR_SINGULAR;

    /* R^2 = num^2 / (den * ss_yy)
     * scaled by (1 << shift)        */
    *result = (int32_t)(
        ((num / den) * num << lr->shift)
        / ss_yy);

    /* Clamp to [0, 1<<shift] */
    if (*result > (int32_t)(1<<lr->shift))
        *result = (int32_t)(1<<lr->shift);
    if (*result < 0) *result = 0;
```

Internal Helper: Slope Decomposition

slope_parts — shared by slope, intercept, predict, r2

```c
static int slope_parts(const struct tiku_kits_ml_linreg *lr,
                       int64_t *num, int64_t *den)
{
    int64_t d;

    if (lr->n < 2)
        return TIKU_KITS_ML_ERR_SIZE;

    *num = (int64_t)lr->n * lr->sum_xy - lr->sum_x * lr->sum_y;

    d = (int64_t)lr->n * lr->sum_xx - lr->sum_x * lr->sum_x;
    if (d == 0)
        return TIKU_KITS_ML_ERR_SINGULAR;    /* all x values identical */

    *den = d;
    return TIKU_KITS_ML_OK;
}
```

Why a Helper?

All four query functions (`slope`, `intercept`, `predict`, `r2`) need the same numerator and denominator decomposition. Factoring it out avoids code duplication and ensures consistent error handling.

Example 1: Sensor Calibration

examples/example_ml_linreg.c — ADC to temperature

```c
struct tiku_kits_ml_linreg lr;
int32_t slope_q, intercept_q, r2_q;
tiku_kits_ml_elem_t y_pred;

tiku_kits_ml_linreg_init(&lr, 8);

/* Calibration data: (raw_adc, temperature_deci_C) */
tiku_kits_ml_linreg_push(&lr, 100, 0);      /* 0.0 C */
tiku_kits_ml_linreg_push(&lr, 200, 100);    /* 10.0 C */
tiku_kits_ml_linreg_push(&lr, 300, 200);    /* 20.0 C */
tiku_kits_ml_linreg_push(&lr, 400, 300);    /* 30.0 C */
tiku_kits_ml_linreg_push(&lr, 500, 400);    /* 40.0 C */

tiku_kits_ml_linreg_slope(&lr, &slope_q);        /* 256 (1.0 Q8) */
tiku_kits_ml_linreg_intercept(&lr, &intercept_q); /* -25600 (-100.0 Q8) */
tiku_kits_ml_linreg_r2(&lr, &r2_q);              /* 256 (1.0 Q8, perfect) */

/* Convert a new ADC reading to temperature */
tiku_kits_ml_linreg_predict(&lr, 350, &y_pred);  /* y_pred = 250 (25.0 C) */
```

Example 2: Trend Detection

Detect drift in sensor readings

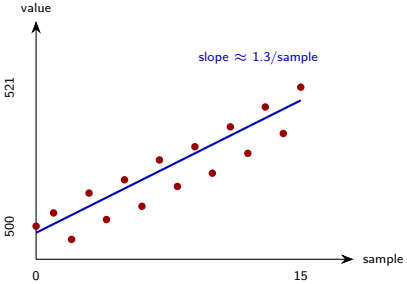
```
struct tiku_kits_ml_linreg lr;
int32_t slope_q;

static const tiku_kits_ml_elem_t
    readings[] = {
    500, 502, 498, 505, 501, 507,
    503, 510, 506, 512, 508, 515,
    511, 518, 514, 521
};

tiku_kits_ml_linreg_init(&lr, 8);
for (i = 0; i < 16; i++)
    tiku_kits_ml_linreg_push(&lr,
        (tiku_kits_ml_elem_t)i,
        readings[i]);

tiku_kits_ml_linreg_slope(&lr, &slope_q);

if (slope_q > 25)        /* > ~0.1/sample */
    alert_upward_drift();
else if (slope_q < -25)
    alert_downward_drift();
```



Use Case

Detects gradual sensor drift (e.g. temperature creep, battery discharge) from a stream of readings. A slope near zero = stable; non-zero = drifting.

Example 3: Battery Charge Prediction

Map battery voltage (mV) to charge level (percent $\times$ 10)

```
struct tiku_kits_ml_linreg lr;
tiku_kits_ml_elem_t y_pred;

tiku_kits_ml_linreg_init(&lr, 8);

/* Calibration: voltage -> charge*10 */
tiku_kits_ml_linreg_push(&lr, 3000, 0);      /* 0% */
tiku_kits_ml_linreg_push(&lr, 3300, 250);    /* 25% */
tiku_kits_ml_linreg_push(&lr, 3600, 500);    /* 50% */
tiku_kits_ml_linreg_push(&lr, 3900, 750);    /* 75% */
tiku_kits_ml_linreg_push(&lr, 4200, 1000);   /* 100% */

/* Predict charge for measured voltage */
tiku_kits_ml_linreg_predict(&lr, 3450, &y_pred);  /* 375 -> 37.5% */
tiku_kits_ml_linreg_predict(&lr, 3750, &y_pred);  /* 625 -> 62.5% */
```

Practical Note

Real battery discharge curves are non-linear. Linear regression provides a reasonable **first approximation** for the near-linear mid-range (20–80%) and is sufficient for many embedded applications where accuracy within 5% is acceptable.

Test Cases Overview

#	Test Function	What It Verifies
1	`test_kits_ml_linreg_perfect_fit`	$y = 2x$: slope = 512 (2.0 Q8), intercept = 0
2	`test_kits_ml_linreg_intercept`	$y = 2x + 1$: slope = 512, intercept = 256 (1.0 Q8)
3	`test_kits_ml_linreg_fractional_slope`	$y = 1.5x + 10$: slope = 384 (1.5 Q8), intercept = 2560
4	`test_kits_ml_linreg_predict`	Prediction at training, interpolation, and extrapolation points
5	`test_kits_ml_linreg_r2`	Perfect fit $\rightarrow R^2 = 256$; noisy data $\rightarrow 0 < R^2 < 256$
6	`test_kits_ml_linreg_negative_values`	$y = -2x + 100$: negative slope and large intercept
7	`test_kits_ml_linreg_reset`	Reset clears data; refit produces correct new model
8	`test_kits_ml_linreg_edge_cases`	1 point $\rightarrow$ ERR_SIZE; identical $x \rightarrow$ SINGULAR; constant y
9	`test_kits_ml_linreg_null_inputs`	All functions reject NULL pointers with ERR_NULL

Test Deep Dive: Perfect Fit & Edge Cases

Perfect fit: $y = 2x$

```c
void test_kits_ml_linreg_perfect_fit(void)
{
    struct tiku_kits_ml_linreg lr;
    int32_t slope_q;

    TEST_GROUP_BEGIN("LinReg Perfect Fit");

    tiku_kits_ml_linreg_init(&lr, 8);
    tiku_kits_ml_linreg_push(&lr, 1, 2);
    tiku_kits_ml_linreg_push(&lr, 2, 4);
    tiku_kits_ml_linreg_push(&lr, 3, 6);

    tiku_kits_ml_linreg_slope(
        &lr, &slope_q);
    TEST_ASSERT(slope_q == 512,
        "slope(y=2x) == 512 (2.0 Q8)");

    TEST_GROUP_END("LinReg Perfect Fit");
}
```

Edge cases: singular data

```c
void test_kits_ml_linreg_edge_cases(void)
{
    struct tiku_kits_ml_linreg lr;
    int32_t slope_q;
    int rc;

    TEST_GROUP_BEGIN("LinReg Edge Cases");

    /* Single point: too few */
    tiku_kits_ml_linreg_init(&lr, 8);
    tiku_kits_ml_linreg_push(&lr, 5, 10);
    rc = tiku_kits_ml_linreg_slope(
        &lr, &slope_q);
    TEST_ASSERT(
        rc == TIKU_KITS_ML_ERR_SIZE,
        "slope with 1 point rejected");

    /* Identical x: singular */
    tiku_kits_ml_linreg_reset(&lr);
    tiku_kits_ml_linreg_push(&lr, 5, 10);
    tiku_kits_ml_linreg_push(&lr, 5, 20);
    rc = tiku_kits_ml_linreg_slope(
        &lr, &slope_q);
    TEST_ASSERT(
```

ML Kit Files

Layer	File	Role
Common	`tikukits/ml/tiku_kits_ml.h`	Shared return codes for all ML sul modules
Linear Regression	`tikukits/ml/regression/tiku_kits_ml_linreg.h` `tikukits/ml/regression/tiku_kits_ml_linreg.c`	API, struct, configuration macros Implementation (43 bytes state, O(1 push)
Tests	`tikukits/tests/ml/test_kits_ml.h` `tikukits/tests/ml/test_kits_ml_linreg.c`	Test function declarations 9 test groups, 22+ assertions
Examples	`tikukits/examples/example_ml_linreg.c`	3 practical examples (calibration, trend prediction)

Convention for New ML Sub-Modules

1. Place files in `ml/<category>/tiku_kits_ml_<module>.h/.c`
2. Include `../tiku_kits_ml.h` for shared return codes

Summary

What We Built

- Streaming OLS linear regression
- Integer-only / fixed-point arithmetic
- 43 bytes of state, no heap
- O(1) per sample push
- On-demand slope, intercept, predict, R^2

Key Design Choices

- `int64_t` accumulators prevent overflow
- Configurable element type and Q-shift
- `predict()` returns original-scale values
- Follows TikuKits naming and structure conventions

Use Cases

- Sensor calibration (ADC $\rightarrow$ physical units)
- Trend detection (drift monitoring)
- Battery charge estimation
- Anomaly flagging via R^2 monitoring

Test Coverage

9 test groups covering: perfect fit, non-zero intercept, fractional slope, prediction, R^2, negative values, reset, edge cases, and NULL safety.

Future ML Sub-Modules

k-NN classifier, decision tree, perceptron — same

Thank You

TikuKits ML: Linear Regression

Integer-Only Machine Learning for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Common:	`tikukits/ml/tiku_kits_ml.h`
Header:	`tikukits/ml/regression/tiku_kits_ml_linreg.h`
Source:	`tikukits/ml/regression/tiku_kits_ml_linreg.c`
Tests:	`tikukits/tests/ml/test_kits_ml_linreg.c`
Examples:	`tikukits/examples/example_ml_linreg.c`